

FOLHA 00 / 09 — CAPA

ANÁLISE E DESENVOLVIMENTO DE SISTEMAS ·
DESENVOLVIMENTO WEB 2 (SPODWE2)

Por Dentro do JavaScript.

Você já escreve JavaScript. Esta aula abre o capô: de onde a linguagem veio, como o escopo e as closures realmente funcionam, o que mudou de ES5 para ES6+ e como separar código bom de código que só parece funcionar.



CARGA HORÁRIA

FORMATO

3h45 · 5 aulas

Encontro único (seg., noturno)

PRÉ-REQUISITOS

Já programa · revisão de HTML/CSS feita

COMO NAVEGAR

Use os números à esquerda ou role a página

FOLHA 01 / 09 — HISTÓRICO

Três nomes que confundem todo mundo

Antes de qualquer linha de código, é preciso separar três coisas que os iniciantes tratam como sinônimos — e que já te confundiram alguma vez. Clique em cada termo:

[JavaScript](#)[· ECMAScript](#)[Runtime / Motor](#)

JAVASCRIPT

A linguagem de programação em si — a implementação prática que você escreve. Nasceu em 1995, criada por Brendan Eich na Netscape em cerca de dez dias. O nome foi uma jogada de marketing para pegar carona no Java, então em alta; as duas linguagens são parentes distantes, não a mesma coisa.

A linha do tempo essencial

1995

NASCIMENTO

Criada na Netscape. Passou por "Mocha" e "LiveScript" antes de virar JavaScript.

2009

ES5

Base consolidada: strict mode, JSON nativo, métodos de array.

2015

ES6 / ES2015

A grande virada: let/const, arrow, classes, módulos, promises.

2016+

ANUAL

Uma versão por ano: async/await, optional chaining, e por aí vai.

Fixe isto: ECMAScript é a *especificação* (o documento, mantido pelo comitê TC39). JavaScript é a *implementação*. O *runtime* (V8 no Chrome/Node, SpiderMonkey no Firefox) é o programa que executa seu código.

Teste rápido: quando você roda `node script.js`, quem executa o código?

A especificação ECMAScript

O motor V8 (runtime)

A linguagem JavaScript

FOLHA 02 / 09 — CARACTERÍSTICAS

O que torna o JavaScript, JavaScript

Cinco características definem o comportamento da linguagem — e explicam boa parte dos bugs que você já viu. Vale reconhecê-las com nome e sobrenome:

CARACTERÍSTICA	O QUE SIGNIFICA NA PRÁTICA
Tipagem dinâmica e fraca	Variáveis não declaram tipo, e a linguagem faz coerção automática entre tipos. Cômico — e traiçoeiro.
Multiparadigma	Aceita estilo imperativo, funcional (funções são valores) e orientado a objetos (baseado em protótipos).
Single-thread + event loop	Roda em uma única thread. Operações assíncronas entram em uma fila e são processadas pelo event loop.
Interpretada com JIT	Motores modernos compilam trechos "quentes" em tempo de execução para ganhar desempenho.
Ubíqua	Navegador, servidor (Node/Deno/Bun), desktop, mobile e até dispositivos embarcados.

A coerção de tipos em ação

A tipagem fraca é a origem de muitos "por que isso deu isso?". Antes de rodar, tente prever cada resultado:

```
// coerção implícita — o JS "converte" tipos sem avisar
console.log(0 == "");    // true ← coerção
console.log(0 === "");   // false ← comparação estrita
console.log([] + []);    // "" ← string vazia
console.log([] + {});    // "[object Object]"
console.log(typeof NaN); // "number" (!)
```

Sem rodar: o que imprime `console.log("5" - 2)` e `console.log("5" + 2)`?

3 e "52"

"52" e 7

3 e 7

Regra de ouro: use sempre `===` e `!==`. As comparações estritas não fazem coerção — elas comparam valor e tipo, eliminando uma classe inteira de bugs silenciosos.

FOLHA 03 / 09 — ESCOPO & HOISTING

Onde suas variáveis realmente vivem

Escopo é a região do código onde uma variável é visível. A confusão clássica vem da diferença entre `var` (escopo de função) e `let/const` (escopo de bloco). Passe pelo simulador abaixo passo a passo:

`var` — escopo de função`let` — escopo de bloco

hoisting & TDZ

```
function teste() {
  if (true) {
    var x = 10;
  }
}
```

ESCOPO DA FUNÇÃO
TESTE()
entrando na
função...

```
console.log(x);  
}
```

var tem escopo de FUNÇÃO. Vamos ver o que acontece com uma variável declarada dentro de um if.

[< Anterior](#)

Passo 1/3

[Próximo >](#)

Pegadinha do var: como ele ignora o bloco, um var declarado dentro de um if ou for "vaza" para toda a função. É por isso que o laço com var e setTimeout imprime o mesmo número várias vezes — todos compartilham a mesma variável.

O que imprime? `console.log(x);` `var x = 10;`

10

undefined

ReferenceError

FOLHA 04 / 09 — FUNÇÕES

Funções são valores de primeira classe

Em JavaScript, uma função é apenas mais um valor: pode ser guardada em variável, passada como argumento e devolvida por outra função. É isso que habilita callbacks, funções de ordem superior e todo o estilo funcional.

// função guardada em variável

```
const dobrar = n => n * 2;
```

// função de ordem superior: recebe outra função

```
const aplicar = (fn, valor) => fn(valor);  
console.log(aplicar(dobrar, 21)); // 42
```

Declaração × expressão × arrow

Três formas de criar funções, com diferenças que importam. Alterne entre elas:

function declarada

function expressão

arrow function

// hoisted por inteiro: pode ser chamada ANTES da declaração
saudar("Ana"); // funciona

```
function saudar(nome) {  
  return `Olá, ${nome}`;  
}
```

A declaração inteira é içada (hoisted). Tem `this` próprio, que depende de *como* a função é chamada.

Quando usar cada uma: arrow para callbacks curtos e para preservar o `this`; `function` quando você precisa de `this` dinâmico (métodos, construtores) ou de hoisting.

FOLHA 05 / 09 — CLOSURES

Closure: a função que lembra

Uma **closure** acontece quando uma função interna continua acessando as variáveis do escopo onde foi criada — *mesmo depois* que a função externa já retornou. É o conceito mais poderoso (e mais cobrado) da linguagem. Avance passo a passo:

CHAMANDO CRIARCONTADOR()

cria um novo escopo com **contador = 0**

1. `criarContador()` é chamada. Dentro dela nasce a variável privada `contador = 0`.

[< Anterior](#)

Passo 1/5

[Próximo >](#)

```
function criarContador() {  
  let contador = 0;    // "privada": ninguém de fora enxerga  
  return function() {  
    contador++;  
    return contador;  
  };  
}
```

```
const proximo = criarContador();  
console.log(proximo()); // 1  
console.log(proximo()); // 2 ← o estado sobreviveu!
```

Para que serve na vida real: encapsular estado privado (como o `contador`), memoização (cache de resultados), fábricas de funções, e manter configuração dentro de callbacks. Toda vez que você usa um módulo com estado interno, closures estão por baixo.

Dois contadores: `const a = criarContador();`
`const b = criarContador();` — eles compartilham o mesmo `contador`?

Sim, é a mesma variável

Não, cada um tem o seu

FOLHA 06 / 09 — EVOLUÇÃO DA LINGUAGEM

De ES5 a ES6+ : o que mudou de verdade

Como você já programa, o mais útil aqui não é a lista de recursos, e sim ver o mesmo código *antes e depois*. Escolha um recurso:

let / const

template literals

destructuring

spread / rest

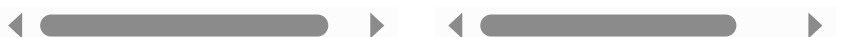
async / await

ES5 — ANTES

ES6+ — DEPOIS

```
var nome = "Ana";
var nome = "Bia"; // redeclara!
if (true) {
  var x = 1;
}
console.log(x); // 1 (vazou)
```

```
const nome = "Ana";
// const nome = "Bia" → erro
if (true) {
  let x = 1;
}
// console.log(x) → ReferenceError
```



let/const trazem escopo de bloco e impedem redeclaração acidental. Prefira const por padrão; let só quando houver reatribuição.

Refinamentos que valem ouro (ES2016+)

// optional chaining (?) e nullish coalescing (??) — ES2020

```
const cidade = usuario?.endereco?.cidade ?? "não informada";
```

// evita erro se 'endereco' for null/undefined, e só cai no default

// quando o valor é null ou undefined (não quando é 0 ou "")

VERSÃO

DESTAQUES

JS ·
FUNDAMENTOS

00

01

02

03

04

05

ES2016	Operador <code>**</code> , <code>Array.includes</code>
ES2017	<code>async/await</code> , <code>Object.entries/values</code>
ES2020	<code>?.</code> , <code>??</code> , <code>BigInt</code> , <code>Promise.allSettled</code>
ES2021+	<code>replaceAll</code> , <code>??=</code> , <code>Array.at</code> , incrementos anuais

FOLHA 07 / 09 — BOAS & MÁS PRÁTICAS

Código que funciona × código que se mantém

Para quem já programa, este é o bloco de maior valor. Cada par abaixo mostra a versão que "funciona por acaso" ao lado da versão que você quer ver num code review.

✗ EVITE

```
var x = 10;
function f(a) {
  return a == "10"
    ? total = a * x
    : null;
}
```

✓ PREFIRA

```
const FATOR = 10;
function calcularTotal(valor) {
  const n = Number(valor);
  return n === FATOR
    ? n * FATOR
    : null;
}
```



Os erros que mais aparecem

MÁ PRÁTICA	POR QUÊ	NO LUGAR, USE
<code>var</code>	Escopo de função e hoisting confuso	<code>const</code> por padrão, <code>let</code> se reatribuir
<code>==</code>	Coerção implícita silenciosa	<code>===</code> e conversão explícita
Variável global	Poluição de namespace,	Encapsular em módulos/funções

acoplamento

Mutar parâmetros	Efeitos colaterais escondidos	Retornar novos valores
Números mágicos	Difícil de ler e manter	Constantes nomeadas
Ignorar erros de <code>await</code>	Falhas silenciosas	<code>try/catch</code> em código <code>async</code>

Automatize a disciplina: configure **ESLint** (encontra problemas) e **Prettier** (padroniza o formato). Boa parte do "estilo" deixa de ser discussão de code review e vira regra automática.

Convenções de nomenclatura

Nome não é decoração: é a primeira documentação que o leitor encontra. A comunidade JavaScript convergiu para três padrões de escrita, cada um com um papel bem definido. Clique para ver onde cada um se aplica:

`camelCase` · `PascalCase` ·

`UPPER_SNAKE_CASE`

CAMELCASE

Primeira palavra minúscula, as seguintes com inicial maiúscula. É o padrão para **variáveis, atributos e métodos** — tudo que é "instância" ou ação. Ex.: `nomeCompleto`, `calcularTotal()`, `this.saldoAtual`.

ELEMENTO	CONVENÇÃO	EXEMPLO
Variável / atributo	<code>camelCase</code>	<code>precoUnitario</code> , <code>this.dataPedido</code>

Método / função	camelCase, começando por verbo	<code>calcularTotal()</code> , <code>buscarUsuario()</code>
Classe	PascalCase	<code>CarrinhoDeCompras</code> , <code>ContaBancaria</code>
Constante fixa	UPPER_SNAKE_CASE	<code>TAXA_JUROS</code> , <code>URL_API</code>
Booleano	prefixo <code>is/has/can</code>	<code>isAtivo</code> , <code>hasPermissao</code>
"Privado" (convenção)	prefixo <code>#</code> (real) ou <code>_</code> (sinal)	<code>#saldo</code> , <code>_cache</code>

Estilo de classe, método e atributo

Reunindo tudo em uma classe bem escrita — repare em cada convenção aplicada ao mesmo tempo:

```
// Classe: PascalCase
class ContaBancaria {
  // Constante da classe: UPPER_SNAKE_CASE
  static TAXA_SAQUE = 2.5;

  // Atributo privado real (#) + camelCase
  #saldo = 0;

  constructor(titular, saldoInicial = 0) {
    this.titular = titular; // atributo público: camelCase
    this.#saldo = saldoInicial;
  }

  // Método: camelCase começando por verbo
  depositar(valor) {
    if (valor <= 0) throw new Error("Valor inválido");
    this.#saldo += valor;
  }

  // Método booleano de leitura: prefixo "tem/pode"
  podeSacar(valor) {
    return valor > 0 && valor <= this.#saldo;
  }
}
```

// *getter* expõe leitura sem expor o atributo

```
get saldo() {
  return this.#saldo;
}
```

```
const conta = new ContaBancaria("Ana", 100);
conta.depositar(50);
console.log(conta.saldo); // 150
console.log(conta.#saldo); // SyntaxError: campo privado
```

Por que # e não _? O prefixo `_saldo` é apenas um *combinado social* — nada impede o acesso de fora. Já `#saldo` é privacidade *real* da linguagem (ES2022): o motor bloqueia o acesso externo. Use `#` quando quiser encapsulamento de verdade.

Padrão de formatação

Formatação consistente reduz a carga cognitiva: o olho para de gastar energia com o "visual" e foca na lógica. As regras mais aceitas na comunidade JS:

✗ FORMATAÇÃO INCONSISTENTE

```
class conta_bancaria{
  Depositar( valor ){
    if(valor>0)
    this.Saldo=this.Saldo+valor}
  }
```

✓ FORMATAÇÃO PADRONIZADA

```
class ContaBancaria {
  depositar(valor) {
    if (valor > 0) {
      this.saldo += valor;
    }
  }
}
```

REGRA

CONVENÇÃO COMUM

Indentação

2 espaços (padrão JS), consistente no projeto

Ponto e vírgula

Sempre ao fim da instrução (evita surpresas do ASI)

Aspas	Um só estilo — aspas simples ' ou template `
Chave {	Na mesma linha da declaração (estilo K&R)
Espaçamento	Espaço após <code>if</code> , <code>for</code> , e ao redor de operadores
Blocos	Sempre com { }, mesmo para uma linha

Na prática: não decore essas regras — deixe o **Prettier** aplicá-las ao salvar e o **ESLint** cobrar as convenções de nomenclatura. O que importa é a turma *reconhecer* o padrão e manter **consistência**: um projeto com um estilo só, do começo ao fim.

Sua checklist de revisão

Marque conforme revisar seu próprio código:

- ☐ Usei `const` por padrão e `let` só onde há reatribuição — nenhum `var`
- ☐ Todas as comparações usam `===` / `!==`
- ☐ Funções pequenas, com nome descritivo e uma responsabilidade
- ☐ Nenhuma variável global desnecessária; estado encapsulado
- ☐ Constantes nomeadas no lugar de números/strings mágicos
- ☐ Código assíncrono trata erros com `try/catch`
- ☐ Classes em `PascalCase`; métodos e atributos em `camelCase`; constantes em `UPPER_SNAKE_CASE`
- ☐ Formatação consistente (indentação, aspas e chaves no mesmo estilo em todo o projeto)

☐ ESlint + Prettier configurados no projeto

FOLHA 08 / 09 — MÓDULOS

Organizando o código em módulos

Módulos dividem o código em arquivos com responsabilidades claras, expondo **apenas o necessário**. Cada módulo tem seu próprio escopo — nada vaza para o global (o mesmo princípio das closures, agora no nível de arquivo).

MATEMATICA.JS — EXPORTA

```
export const PI = 3.14159;

export function areaCirculo(r) {
  return PI * r * r;
}

// um export default por arquivo
export default function saudac
  return "Olá, módulos!";
}
```

MAIN.JS — IMPORTA

```
// default sem chaves; nomeado
import saudacao, { PI, areaCirculo
  from "./matematica.js";

console.log(areaCirculo(2)); // 12.56637
console.log(saudacao()); // "Olá, módulos!"
```

ES Modules × CommonJS

	ES MODULES (MODERNO)	COMMONJS (NODE CLÁSSICO)
Exportar	export / export default	module.exports
Importar	import ... from	require()
Onde	Navegador (<script type="module">) e Node moderno	Node tradicional

Carregamento

Estático (analísável
antes de executar)Dinâmico (em
tempo de
execução)

No navegador: para usar `import/export` direto no HTML, o script precisa ser declarado como `<script type="module">`. Sem isso, o navegador trata como script clássico e o `import` falha.

FOLHA 09 / 09 — EXERCÍCIOS

Fixação

Resolva no seu editor (navegador com DevTools ou Node.js). Tente antes de abrir o gabarito.

1. Preveja a saída (escopo & hoisting) **BÁSICO** +
2. Conserte a coerção **BÁSICO** +
3. Escreva uma closure **INTERMEDIÁRIO** +
4. Modernize para ES6+ **INTERMEDIÁRIO** +
5. Projete um módulo (síntese) **AVANÇADO** +

ENCERRAMENTO

Para aprofundar

- MDN Web Docs — JavaScript
(developer.mozilla.org)
- Especificação e propostas ECMAScript (tc39.es)
- Node.js para rodar exemplos fora do navegador
(nodejs.org)
- Ferramentas de disciplina: ESLint e Prettier

Desenvolvimento Web 2 (SPODWE2) · Aula de fundamentos
de JavaScript · 3h45 / 5 aulas